In [1]:
```
# ============================================================
# 12. RECOMMENDED TRAINING PROMPT
# ============================================================

SYSTEM_PROMPT = """
You are a retail analytics assistant.

Rules:
- Never invent numbers.
- Use only provided metrics.
- Give concise analytical insights.
- Mention exact values.
- Focus on business interpretation.
"""
```

# Chia dữ liệu cho TRAIN + VALIDATION + TEST

In [2]:
```
#### Traning script

import torch
import os

from datasets import load_dataset

from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    EarlyStoppingCallback
)

from peft import (
    LoraConfig,
    get_peft_model
)

from trl import SFTTrainer

import matplotlib.pyplot as plt


# ============================================================
# CONFIG
# ============================================================

# BASE_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"

# DATASET_PATH = "./Training_Data/tri_objective_qa_dataset.jsonl"

# OUTPUT_DIR = "./Training_Data/fine_tune_tri_objective_model_" + d
```

```
BASE_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
DB_name = "Retail"
DATASET_PATH = "./Training_Data/FI_qa_dataset_Retail.jsonl"

OUTPUT_DIR = "./Training_Data/fine_tune_Qwen2.5-1.5B_Dataset_" + DB

# ================================================================
# LOAD TOKENIZER
# ================================================================

tokenizer = AutoTokenizer.from_pretrained(
    BASE_MODEL,
    trust_remote_code=True
)

tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
# ================================================================
# QUANTIZATION
# ================================================================



# ================================================================
# LOAD MODEL
# ================================================================

if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
print(f"Using device: {device}")

model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL
    # dtype=torch.float32
)

model.to(device)

# ================================================================
# PREPARE FOR LoRA
# ================================================================


# ================================================================
# LoRA CONFIG
# ================================================================

lora_config = LoraConfig(
    r=32,
    lora_alpha=64,
    target_modules=[
```

```
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj"
    ],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)
model.enable_input_require_grads()
model.gradient_checkpointing_enable()
model.config.use_cache = False
model.print_trainable_parameters()
model.config.pad_token_id = tokenizer.pad_token_id

# ============================================================
# LOAD DATASET
# ============================================================

dataset = load_dataset(
    "json",
    data_files=DATASET_PATH,
    split="train"
)


# Shuffle để tránh dữ liệu theo thứ tự
dataset = dataset.shuffle(seed=42)

# Chia Train (80%) và phần còn lại (20%)
train_test = dataset.train_test_split(
    test_size=0.2,
    seed=42
)

train_dataset = train_test["train"]
temp_dataset = train_test["test"]

# Chia tiếp Validation (10%) và Test (10%)
valid_test = temp_dataset.train_test_split(
    test_size=0.5,
    seed=42
)

valid_dataset = valid_test["train"]
test_dataset = valid_test["test"]

print(f"Train:      {len(train_dataset)}")
print(f"Validation: {len(valid_dataset)}")
print(f"Test:       {len(test_dataset)}")
```

```
# ============================================================
# FORMAT DATASET
# ============================================================

def format_sample(sample):

    text = f"""
### Instruction:
{sample['instruction']}

### Input:
{sample['input']}

### Response:
{sample['output']}
"""

    return {"text": text}


train_dataset = train_dataset.map(format_sample)
valid_dataset = valid_dataset.map(format_sample)
test_dataset = test_dataset.map(format_sample)


# ============================================================
# SAVE DATASETS
# ============================================================

import os

train_path = "./Training_Data/train.json"
valid_path = "./Training_Data/validation.json"
test_path = "./Training_Data/test.json"

if (
    os.path.exists(train_path)
    and os.path.exists(valid_path)
    and os.path.exists(test_path)
):
    print("Dataset files already exist. Skip splitting.")

else:
    print("Creating dataset split...")

    train_dataset.to_json(train_path)
    valid_dataset.to_json(valid_path)
    test_dataset.to_json(test_path)

    print("Dataset saved successfully.")
```

```
/opt/anaconda3/envs/LLM/lib/python3.11/site-packages/tqdm/auto.py:2
1: TqdmWarning: IProgress not found. Please update jupyter and ipywi
dgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.
html
  from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub. Ple
ase set a HF_TOKEN to enable higher rate limits and faster download
s.
Using device: mps
Loading weights: 100%|██████████| 338/338 [00:00<00:00, 26740.51it/
s]
trainable params: 36,929,536 || all params: 1,580,643,840 || trainab
le%: 2.3364
Train:      964
Validation: 121
Test:       121
Dataset files already exist. Skip splitting.
```

Load base model Qwen2.5-1.5B-Instruction

In [16]:

```python
import json
import torch
import pandas as pd

from tqdm import tqdm
from transformers import AutoTokenizer
from transformers import AutoModelForCausalLM
from bert_score import score

# ============================================================
# CONFIG
# ============================================================

MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"

TEST_FILE = "./Training_Data/test.json"

OUTPUT_CSV = "./Training_Data/Baseline_Qwen2.5-1.5B-instruction_BER

MAX_NEW_TOKENS = 256

DEVICE = "cuda" if torch.cuda.is_available() else \
         "mps" if torch.backends.mps.is_available() else \
         "cpu"

print("Device:", DEVICE)

# ============================================================
# LOAD MODEL
# ============================================================

tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME,
    trust_remote_code=True
)
```

```
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype="auto",
    device_map="auto",
    trust_remote_code=True
)

model.eval()

# ============================================================
# LOAD DATASET
# ============================================================

dataset = []

with open(TEST_FILE, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line:
            dataset.append(json.loads(line))

print("Loaded:", len(dataset))

predictions = []
references = []

# ============================================================
# GENERATE
# ============================================================

for sample in tqdm(dataset):

    prompt = f"""Instruction:
{sample["instruction"]}

Input:
{sample["input"]}

Answer:
"""

    messages = [
        {
            "role": "user",
            "content": prompt
        }
    ]

    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    inputs = tokenizer(
```

```
        text,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():

        outputs = model.generate(
            **inputs,
            max_new_tokens=MAX_NEW_TOKENS,
            temperature=0,
            do_sample=False,
            eos_token_id=tokenizer.eos_token_id,
            pad_token_id=tokenizer.pad_token_id
        )

    generated = outputs[0][inputs.input_ids.shape[1]:]

    prediction = tokenizer.decode(
        generated,
        skip_special_tokens=True
    ).strip()

    predictions.append(prediction)

    references.append(sample["output"])

# ====================================================
# COMPUTE BERT SCORE
# ====================================================

P, R, F1 = score(
    predictions,
    references,
    lang="en",
    verbose=True
)



# ====================================================
# COMPUTE ROUGE-L
# ====================================================
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(
    ["rougeL"],
    use_stemmer=True
)

rouge_precision = []
rouge_recall = []
rouge_f1 = []

for pred, ref in zip(predictions, references):

    scores = scorer.score(ref, pred)
```

```
    rouge = scores["rougeL"]

    rouge_precision.append(rouge.precision)
    rouge_recall.append(rouge.recall)
    rouge_f1.append(rouge.fmeasure)

# ============================================================
# SAVE
# ============================================================

df = pd.DataFrame({

    "instruction":[x["instruction"] for x in dataset],

    "input":[x["input"] for x in dataset],

    "ground_truth":references,

    "prediction":predictions,

    "bert_precision":P.numpy(),

    "bert_recall":R.numpy(),

    "bert_f1":F1.numpy(),

    "rougeL_precision":rouge_precision,

    "rougeL_recall":rouge_recall,

    "rougeL_f1":rouge_f1

})

df.to_csv(
    OUTPUT_CSV,
    index=False
)

print(df.head())

print()

print("="*60)

print("Average BERTScore")

print(f"Precision : {P.mean():.4f}")

print(f"Recall    : {R.mean():.4f}")

print(f"F1         : {F1.mean():.4f}")

print()

print("=" * 60)
```

```
print("Average ROUGE-L")

print(f"Precision : {sum(rouge_precision)/len(rouge_precision):.4f}
print(f"Recall    : {sum(rouge_recall)/len(rouge_recall):.4f}")
print(f"F1         : {sum(rouge_f1)/len(rouge_f1):.4f}")
```

```
Device: mps
Loading weights: 100%|██████████| 338/338 [00:00<00:00, 70223.64it/
s]
Loaded: 121
100%|██████████| 121/121 [08:41<00:00,  4.31s/it]
Loading weights: 100%|██████████| 389/389 [00:00<00:00, 7164.93it/s]
[transformers] RobertaModel LOAD REPORT from: roberta-large
Key                        | Status     |
---------------------------+------------+-
lm_head.layer_norm.bias    | UNEXPECTED |
lm_head.layer_norm.weight  | UNEXPECTED |
lm_head.dense.weight       | UNEXPECTED |
lm_head.dense.bias         | UNEXPECTED |
lm_head.bias               | UNEXPECTED |
pooler.dense.bias          | MISSING    |
pooler.dense.weight        | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/arch
itecture; not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing
from the checkpoint. Consider training on your downstream task.
calculating scores...
computing bert embedding.
100%|██████████| 4/4 [00:12<00:00,  3.03s/it]
computing greedy matching.
100%|██████████| 2/2 [00:00<00:00, 37.58it/s]
```

```
done in 12.19 seconds, 9.93 sentences/sec
                                          instruction  \
0  Analyze the competitive position of itemset [3...
1  Determine if itemset [39, 4444] satisfies freq...
2  Compare the performance of itemset [39, 48, 48...
3  How does itemset [39, 48, 1310] have high perf...
4  Explain whether itemset [48, 7560] is a freque...

                                              input  \
0  Itemset A:\n- Name: [39, 48, 10486]\n- Support...
1  Itemset:\n- Name: [39, 4444]\n- Supports: 73\n...
2  Itemset A:\n- Name: [39, 48, 4879]\n- Supports...
3  Itemset A:\n- Name: [39, 48, 1310]\n- Supports...
4  Itemset:\n- Name: [48, 7560]\n- Supports: 6\n-...

                                         ground_truth  \
0  \n- Both itemsets [39, 48, 10486] and [39, 48,...
1  \n- The itemset [39, 4444] is purchased freque...
2  \n- Both itemsets [39, 48, 4879] and [48, 41, ...
3  \n- Both itemsets [39, 48, 1310] and [39, 48, ...
4  \n- The itemset [48, 7560] is purchased regula...

                                          prediction  bert_precision
\
0  To analyze the competitive position of itemset...        0.852476
1  To determine if the itemset [39, 4444] satisfi...        0.847873
2  To compare the performance of the two itemsets...        0.843446
3  To determine how the itemset [39, 48, 1310] ha...        0.852617
4  To determine if the itemset [48, 7560] is a fr...        0.807795

   bert_recall   bert_f1  rougeL_precision  rougeL_recall  rougeL_f1
0     0.850304  0.851389          0.350649       0.171975   0.230769
1     0.863598  0.855663          0.212500       0.314815   0.253731
2     0.798734  0.820481          0.291139       0.145570   0.194093
3     0.854383  0.853499          0.350649       0.171975   0.230769
4     0.856955  0.831649          0.236559       0.379310   0.291391

============================================================
Average BERTScore
Precision : 0.8362
Recall    : 0.8538
F1        : 0.8447

============================================================
Average ROUGE-L
Precision : 0.2426
Recall    : 0.2782
F1        : 0.2274
```

** Đánh giá Numerical Faithfulness mô hình baseline pre-train Qwen2.5-1.5B-Instruction **

In [21]:
```
def numerical_faithfulness(input_text, prediction_text):

    input_numbers = set(extract_numbers(input_text))
    prediction_numbers = extract_numbers(prediction_text)
```

```
    if len(prediction_numbers) == 0:
        return {
            "InputNumbers": list(input_numbers),
            "PredictionNumbers": [],
            "HallucinatedNumbers": [],
            "CorrectNumbers": 0,
            "PredictionCount": 0,
            "HallucinationCount": 0,
            "NumericalFaithfulness": 1.0
        }

    correct = 0
    hallucinated = []

    for num in prediction_numbers:

        if num in input_numbers:
            correct += 1
        else:
            hallucinated.append(num)

    return {
        "InputNumbers": list(input_numbers),
        "PredictionNumbers": prediction_numbers,
        "HallucinatedNumbers": hallucinated,
        "CorrectNumbers": correct,
        "PredictionCount": len(prediction_numbers),
        "HallucinationCount": len(hallucinated),
        "NumericalFaithfulness": correct / len(prediction_numbers)
    }

# ============================================================
# Evaluation
# ============================================================

results = []

for sample, pred in zip(dataset, predictions):

    result = numerical_faithfulness(
        sample["input"],
        pred
    )

    results.append(result)

df = pd.DataFrame(results)

print("=" * 60)
print("Average Numerical Faithfulness:",
      df["NumericalFaithfulness"].mean())
print("=" * 60)

df.to_csv(
    "./Training_Data/NumericalFaithfulness_Baseline_Qwen2.5-1.5B-In
```

```
    index=False,
    encoding="utf-8-sig"
)
```

```
============================================================
Average Numerical Faithfulness: 0.7840566936924855
============================================================
```

** Explanation completeness **

In [23]:
```
import re

# ============================================================
# Keywords
# ============================================================

SUPPOR_KEYWORDS = [
    "support",
    "supports",
    "support threshold",
    "support exceeds",
    "purchased frequently",
    "frequently occurring",
    "frequent itemset"
]

AVG_LENGTH_KEYWORDS = [
    "average support-transaction length",
    "average transaction length",
    "longer transaction",
    "shorter transaction",
    "longer transactions",
    "shorter transactions",
    "average length"
]

FI_SCORE_KEYWORDS = [

    "fi score",

    "score",

    "overall performance",

    "better performance",

    "higher score",

    "lower score",

    "overall score",

    "business impact",

    "overall impact"
```

```
]


# ============================================================
# Utility
# ============================================================

def contains_keywords(text, keywords):

    text = text.lower()

    return any(keyword.lower() in text for keyword in keywords)


# ============================================================
# Support
# ============================================================

def evaluate_support(prediction):

    return int(
        contains_keywords(
            prediction,
            SUPPOR_KEYWORDS
        )
    )


# ============================================================
# Average Support Length
# ============================================================

def evaluate_average_length(prediction):

    return int(
        contains_keywords(
            prediction,
            AVG_LENGTH_KEYWORDS
        )
    )


# ============================================================
# FI Score
# ============================================================

def evaluate_fi_score(prediction):

    return int(
        contains_keywords(
            prediction,
            FI_SCORE_KEYWORDS
        )
    )
```

```
# ============================================================
# Overall
# ============================================================

def explanation_completeness(prediction):

    support = evaluate_support(prediction)

    avg = evaluate_average_length(prediction)

    fi = evaluate_fi_score(prediction)

    return {
        "support": support,
        "average_length": avg,
        "fi_score": fi,
        "ExplanationCompleteness":
            (support + avg + fi) / 3
    }


import pandas as pd

rows = [explanation_completeness(pred) for pred in predictions]

df = pd.DataFrame(rows)


# print(df.head())

# print()

print("="*50)
print(df.mean())

df.to_csv(
    "./Training_Data/ExplanationCompleteness_Baseline_Qwen2.5-1.5B-
    index=False
)
```

```
==================================================
support                    1.000000
average_length             0.710744
fi_score                   0.669421
ExplanationCompleteness    0.793388
dtype: float64
```

** Numerical correctness **

In [27]:

```
import re
import pandas as pd

# ============================================================
# CONFIG
# ============================================================
```

```
TOLERANCE = 0.1

# ============================================================
# Remove Itemset IDs
# ============================================================

def remove_itemsets(text):
    """
    Remove itemset IDs.

    Example:
        [39, 41, 1799]
        [6009]
    """

    if text is None:
        return ""

    return re.sub(r"\[[^\]]*\]", "", text)


# ============================================================
# Extract Numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all numerical values.
    """

    text = remove_itemsets(text)

    pattern = r"-?\d+(?:\.\d+)?"

    numbers = re.findall(pattern, text)

    return [float(x) for x in numbers]


# ============================================================
# Direct Match
# ============================================================

def is_direct_match(pred, input_numbers):

    for x in input_numbers:

        if abs(pred - x) <= TOLERANCE:
            return True

    return False


# ============================================================
# Division Match
# ============================================================
```

```
def is_division_match(pred, input_numbers):

    n = len(input_numbers)

    for i in range(n):

        for j in range(n):

            if i == j:
                continue

            a = input_numbers[i]
            b = input_numbers[j]

            # if abs(b) <= TOLERANCE:
            #     continue

            ratio = a / b

            if abs(pred - ratio) <= TOLERANCE:
                return True

    return False


# ============================================================
# Numerical Correctness
# ============================================================

def numerical_correctness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)

    pred_numbers = extract_numbers(prediction_text)

    direct_match = 0
    division_match = 0
    incorrect = 0

    for pred in pred_numbers:

        # 1. Có trực tiếp trong input
        if is_direct_match(pred, input_numbers):

            direct_match += 1

        # 2. Không có nhưng bằng phép chia
        elif is_division_match(pred, input_numbers):

            division_match += 1

        # 3. Sai hoàn toàn
        else:

            incorrect += 1
```

```
    correct = direct_match + division_match

    if len(pred_numbers) == 0:
        score = 0
    else:
        score = correct / len(pred_numbers)

    return {

        "input_numbers": len(input_numbers),

        "predicted_numbers": len(pred_numbers),

        "direct_match": direct_match,

        "division_match": division_match,

        "correct": correct,

        "incorrect": incorrect,

        "NumericalCorrectness": score
    }


# ============================================================
# Evaluate
# ============================================================

rows = []

for sample, prediction in zip(dataset, predictions):

    rows.append(
        numerical_correctness(
            sample["input"],
            prediction
        )
    )

df = pd.DataFrame(rows)

print(df)

print("="*70)
print("Average")
print("="*70)
print(df.mean(numeric_only=True))

print()

print("="*70)
print("Percentage")
print("="*70)
```

```
print(f"Average numbers in Input      : {df['input_numbers'].mean()
print(f"Average numbers in Prediction : {df['predicted_numbers'].me
print(f"Direct Match                  : {df['direct_match'].mean():
print(f"Division Match                : {df['division_match'].mean(
print(f"Correct Numbers               : {df['correct'].mean():.4f}"
print(f"Incorrect Numbers             : {df['incorrect'].mean():.4f
print(f"Numerical Correctness         : {df['NumericalCorrectness']

df.to_csv(
    f"./Training_Data/NumericalCorrectness_Baseline_Qwen2.5-1.5B-In
    index=False
)

print("\nSaved to NumericalCorrectness_Baseline_Qwen2.5-1.5B-Instru
```

```
     input_numbers  predicted_numbers  direct_match  division_match
correct  \
0               16                  9             9               0
9
1                8                  4             4               0
4
2               16                 10            10               0
10
3               16                  8             8               0
8
4                8                 11            10               1
11
..             ...                ...           ...             ...
...
116             17                 16            16               0
16
117             16                 10             6               0
6
118              8                  3             3               0
3
119              8                  7             6               0
6
120              8                  7             4               0
4

     incorrect  NumericalCorrectness
0            0              1.000000
1            0              1.000000
2            0              1.000000
3            0              1.000000
4            0              1.000000
..         ...                   ...
116          0              1.000000
117          4              0.600000
118          0              1.000000
119          1              0.857143
120          3              0.571429

[121 rows x 7 columns]
================================================================
==
```

```
Average
================================================================================
==
input_numbers           12.181818
predicted_numbers        9.024793
direct_match             6.900826
division_match           0.504132
correct                  7.404959
incorrect                1.619835
NumericalCorrectness     0.846668
dtype: float64

================================================================================
==
Percentage
================================================================================
==
Average numbers in Input      : 12.1818
Average numbers in Prediction : 9.0248
Direct Match                  : 6.9008
Division Match                : 0.5041
Correct Numbers               : 7.4050
Incorrect Numbers             : 1.6198
Numerical Correctness         : 84.67%

Saved to NumericalCorrectness_Baseline_Qwen2.5-1.5B-Instruction.csv
```

Fine-tuned Qwen2.5-1.5B-Instruction

In [4]:
```python
# ============================================================
# TRAINING ARGUMENTS
# ============================================================

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    seed=42,
    learning_rate=2e-5,

    num_train_epochs=20,
    logging_first_step=True,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    gradient_checkpointing = True,
    optim="adamw_torch",
    weight_decay=0.01,
    lr_scheduler_type="cosine",
    warmup_steps=100,
    max_grad_norm=1.0,
    fp16=False,
    dataloader_pin_memory=False,
    logging_strategy="steps",
    logging_steps=10,

    eval_strategy="epoch",
    save_strategy="epoch",
```

```
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
    greater_is_better=False,

    save_total_limit=2,

    report_to="none"

)

# ============================================================
# TRAINER
# ============================================================

trainer = SFTTrainer(
    model=model,

    train_dataset=train_dataset,

    eval_dataset=valid_dataset,

    args=training_args,

    formatting_func=lambda x: x["text"],
    # dataset_text_field="text",
    # max_seq_length=2048,
    # packing=True,
    callbacks=[
        EarlyStoppingCallback(
            early_stopping_patience=3
        )
    ]
)

# ============================================================
# TRAIN
# ============================================================

import os
from transformers.trainer_utils import get_last_checkpoint
import json
import pandas as pd

last_checkpoint = get_last_checkpoint(OUTPUT_DIR)

if last_checkpoint is not None:
    trainer_state = os.path.join(
        last_checkpoint,
        "trainer_state.json"
        )
    with open(trainer_state, "r") as f:
        state = json.load(f)

    log_history = state["log_history"]
    df_log = pd.DataFrame(log_history)
    cols = [
```

```
    "step",
    "epoch",
    "loss",
    "eval_loss",
    "learning_rate"
    ]

    df_log = pd.DataFrame(log_history)

    df_log = df_log.reindex(columns=cols)

    display(df_log)
    print(f"Resume from: {last_checkpoint}")
    trainer.train(resume_from_checkpoint=last_checkpoint)

else:
    print("Start new training")
    trainer.train()
    log_history = trainer.state.log_history


results = trainer.evaluate(
    eval_dataset=test_dataset
)
print(results)




# ============================================================
# SAVE MODEL
# ============================================================

trainer.save_state()
trainer.save_model()
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
print(f"Fine-tuned {BASE_MODEL} completed!")
```

```
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ
from the model config and generation config. The model config and ge
neration config were aligned accordingly, being updated with the tok
enizer's values. Updated tokens: {'bos_token_id': None, 'pad_token_i
d': 151643}.
Start new training
```

[ 610/1220 2:53:18 < 2:53:52, 0.06 it/s, Epoch 10/20]

| Epoch | Training Loss | Validation Loss | Entropy | Num Tokens | Mean Token Accuracy |
|---|---|---|---|---|---|
| 1 | 0.483509 | 0.420562 | 0.428409 | 546463.000000 | 0.845089 |
| 2 | 0.309741 | 0.300116 | 0.310100 | 1092926.000000 | 0.885285 |
| 3 | 0.267027 | 0.268763 | 0.278597 | 1639389.000000 | 0.896060 |
| 4 | 0.253176 | 0.260188 | 0.266882 | 2185852.000000 | 0.899789 |
| 5 | 0.249069 | 0.257866 | 0.260762 | 2732315.000000 | 0.900315 |
| 6 | 0.237338 | 0.256534 | 0.254245 | 3278778.000000 | 0.904121 |
| 7 | 0.229536 | 0.253413 | 0.250820 | 3825241.000000 | 0.905635 |
| 8 | 0.220237 | 0.256276 | 0.243600 | 4371704.000000 | 0.905917 |
| 9 | 0.206854 | 0.257623 | 0.237530 | 4918167.000000 | 0.907267 |
| 10 | 0.210998 | 0.258705 | 0.236316 | 5464630.000000 | 0.907464 |

```
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.52it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.54it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.53it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.60it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.53it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.57it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.58it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.57it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.54it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.57it/s]
[transformers] There were missing keys in the checkpoint model loade
d: ['lm_head.weight'].
```

[16/16 00:27]

| Training Loss | Validation Loss | Epoch | Entropy | Num Tokens | Mean Token Accuracy |
|---|---|---|---|---|---|
| 0.210998 | 0.261019 | 10 | 0.242494 | 5464630.000000 | 0.906180 |

```
{'eval_loss': 0.261018842458725, 'eval_entropy': 0.2424935270100832,
'eval_num_tokens': 5464630.0, 'eval_mean_token_accuracy': 0.90618049
72589016}
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.61it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.34it/s]
Fine-tuned Qwen/Qwen2.5-1.5B-Instruct completed!
```

In [31]:
```python
import pandas as pd

df = pd.DataFrame(log_history)

# Train loss cuối cùng của mỗi epoch
train_df = (
```

```
    df[df["loss"].notna()]
    .groupby(df[df["loss"].notna()]["epoch"].astype(int))
    .last()
    .reset_index(drop=True)
)

train_df["Epoch"] = train_df["epoch"].astype(int) + 1

train_df = train_df[
    ["Epoch", "loss"]
].rename(
    columns={"loss": "Training Loss"}
)

# Validation loss
eval_df = (
    df[df["eval_loss"].notna()]
    .copy()
)

eval_df["Epoch"] = eval_df["epoch"].astype(int)

eval_df = eval_df[
    ["Epoch", "eval_loss"]
].rename(
    columns={"eval_loss": "Validation Loss"}
)

history = pd.merge(
    train_df,
    eval_df,
    on="Epoch",
    how="outer"
).sort_values("Epoch")

display(history)

train_loss = []
val_loss = []

for log in log_history:
    if "loss" in log:
        train_loss.append((log["step"], log["loss"]))

    if "eval_loss" in log:
        val_loss.append((log["step"], log["eval_loss"]))

train_steps = [x[0] for x in train_loss]
train_values = [x[1] for x in train_loss]

val_steps = [x[0] for x in val_loss]
val_values = [x[1] for x in val_loss]

import matplotlib.pyplot as plt

# ------------------------------------------------------------------
```

```
# Plot loss per epoch
# ----------------------------------------------------------------
plt.figure(figsize=(8, 5))

# Epoch-wise curves (one point per epoch)
plt.plot(history["Epoch"], history["Training Loss"],
         marker='o', linestyle='-', color="#1f77b4", label="Trainin

plt.plot(history["Epoch"], history["Validation Loss"],
         marker='s', linestyle='--', color="#ff7f0e", label="Valida

# ----------  FONT-SIZE SETTINGS ----------
plt.xlabel("Epoch", fontsize=16)
plt.ylabel("Loss", fontsize=16)
plt.title("Training and Validation Loss (per epoch)", fontsize=17)

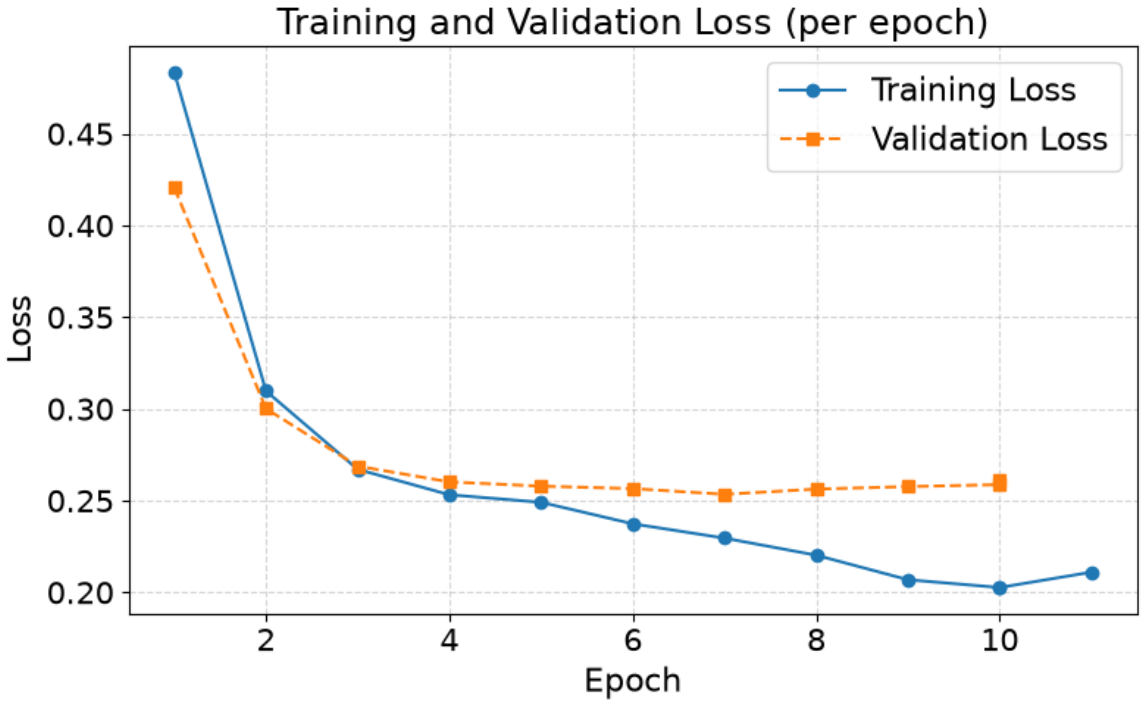
# Tick labels
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)

# Legend font size
plt.legend(fontsize=16, loc="upper right")

# ----------  FINISH ----------
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()

# Save & show
plt.savefig(
    "./Training_Data/loss_plot_Qwen2.5-1.5B-Instruction_epoch.pdf",
    dpi=300,
    bbox_inches="tight"
)
plt.show()
```

| | Epoch | Training Loss | Validation Loss |
|---|---|---|---|
| **0** | 1 | 0.483509 | 0.420562 |
| **1** | 2 | 0.309741 | 0.300116 |
| **2** | 3 | 0.267027 | 0.268763 |
| **3** | 4 | 0.253176 | 0.260188 |
| **4** | 5 | 0.249069 | 0.257866 |
| **5** | 6 | 0.237338 | 0.256534 |
| **6** | 7 | 0.229536 | 0.253413 |
| **7** | 8 | 0.220237 | 0.256276 |
| **8** | 9 | 0.206854 | 0.257623 |
| **9** | 10 | 0.202621 | 0.258705 |
| **10** | 10 | 0.202621 | 0.261019 |
| **11** | 11 | 0.210998 | NaN |




In [6]:
```python
from transformers import pipeline

pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    device=device,   # hoặc device_map="auto" nếu dùng CUDA
)

predictions = []

for sample in test_dataset:
```

```
    prompt = f"""### Instruction:
{sample["instruction"]}

### Input:
{sample["input"]}

### Response:
"""

    result = pipe(
        prompt,
        max_new_tokens=256,
        do_sample=False,
        temperature=0,
        return_full_text=False
    )[0]["generated_text"]

    predictions.append({
        "input": sample["input"],              # <-- thêm dòng này
        "ground_truth": sample["output"],
        "prediction": result.strip()
    })
```

```
[transformers] Passing `generation_config` together with generation-
related arguments=({'max_new_tokens', 'temperature', 'do_sample'}) i
s deprecated and will be removed in future versions. Please pass eit
her a `generation_config` object OR all generation parameters explic
itly, but not both.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE to
kenizer Qwen2Tokenizer. The clean_up_tokenization post-processing st
ep is designed for WordPiece tokenizers and is destructive for BPE (
it strips spaces before punctuation). Set clean_up_tokenization_spac
es=False to suppress this warning, or set clean_up_tokenization_spac
es_for_bpe_even_though_it_will_corrupt_output=True to force cleanup
anyway.
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
```

```
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Both `max_new_tokens` (=256) and `max_length`(=20) se
em to have been set. `max_new_tokens` will take precedence. Please r
efer to the documentation for more information. (https://huggingfac
e.co/docs/transformers/main/en/main_classes/text_generation)
```

BERT SCORE

In [7]:
```python
from bert_score import score

preds = [x["prediction"] for x in predictions]
refs = [x["ground_truth"] for x in predictions]

P, R, F1 = score(
    preds,
    refs,
    lang="en",
    verbose=True
)

print("Precision :", P.mean().item())
print("Recall    :", R.mean().item())
print("F1        :", F1.mean().item())
```

```
Loading weights: 100%|██████████| 389/389 [00:00<00:00, 7470.15it/s]
[transformers] RobertaModel LOAD REPORT from: roberta-large
Key                         | Status     |
----------------------------+------------+-
lm_head.layer_norm.bias     | UNEXPECTED |
lm_head.layer_norm.weight   | UNEXPECTED |
lm_head.dense.weight        | UNEXPECTED |
lm_head.dense.bias          | UNEXPECTED |
lm_head.bias                | UNEXPECTED |
pooler.dense.bias           | MISSING    |
pooler.dense.weight         | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/arch
itecture; not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing
from the checkpoint. Consider training on your downstream task.
```

```
calculating scores...
computing bert embedding.
100%|██████████| 4/4 [00:11<00:00,  2.94s/it]
computing greedy matching.
100%|██████████| 2/2 [00:00<00:00, 42.76it/s]
done in 11.81 seconds, 10.24 sentences/sec
Precision : 0.9207029938697815
Recall    : 0.9275710582733154
F1        : 0.9238168001174927
```

** Đánh giá ROUGE-L sau khi đã fine-tune **

In [15]:

```
# ============================================================
# ROUGE-L
# ============================================================

scorer = rouge_scorer.RougeScorer(
    ["rougeL"],
    use_stemmer=True
)

rouge_p = []
rouge_r = []
rouge_f = []

for pred, ref in zip(preds, refs):

    score_result = scorer.score(ref, pred)["rougeL"]

    rouge_p.append(score_result.precision)
    rouge_r.append(score_result.recall)
    rouge_f.append(score_result.fmeasure)

print("\n" + "=" * 60)
print("Average ROUGE-L")
print(f"Precision : {sum(rouge_p)/len(rouge_p):.4f}")
print(f"Recall    : {sum(rouge_r)/len(rouge_r):.4f}")
print(f"F1         : {sum(rouge_f)/len(rouge_f):.4f}")

# ============================================================
# Save sample-level results
# ============================================================

df = pd.DataFrame({
    "ground_truth": refs,
    "prediction": preds,
    "bert_precision": P.numpy(),
    "bert_recall": R.numpy(),
    "bert_f1": F1.numpy(),
    "rougeL_precision": rouge_p,
    "rougeL_recall": rouge_r,
    "rougeL_f1": rouge_f
})

df.to_csv("./Training_Data/fine-tuned_Qwen2.5-1.5B_BERT-score_ROUGE
```

```
print("\nSaved: fine-tuned_Qwen2.5-1.5B_BERT-score_ROUGE-L.csv")
```

```
============================================================
Average ROUGE-L
Precision : 0.5875
Recall    : 0.5873
F1        : 0.5441

Saved: fine-tuned_Qwen2.5-1.5B_BERT-score_ROUGE-L.csv
```

Numerical Faithfulness

In [8]:
```
import re
import pandas as pd

# ============================================================
# Extract numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all integers and decimal numbers from text.
    """
    return re.findall(r"\d+(?:\.\d+)?", text)


# ============================================================
# Numerical Faithfulness
# ============================================================

def numerical_faithfulness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)
    prediction_numbers = extract_numbers(prediction_text)

    # Prediction không sinh số
    if len(prediction_numbers) == 0:
        return {
            "InputNumbers": input_numbers,
            "PredictionNumbers": prediction_numbers,
            "HallucinatedNumbers": [],
            "CorrectNumbers": 0,
            "PredictionCount": 0,
            "NumericalFaithfulness": 1.0
        }

    correct = 0
    hallucinated = []

    for num in prediction_numbers:

        if num in input_numbers:
            correct += 1
        else:
            hallucinated.append(num)
```

```
    score = correct / len(prediction_numbers)

    return {
        "InputNumbers": input_numbers,
        "PredictionNumbers": prediction_numbers,
        "HallucinatedNumbers": hallucinated,
        "CorrectNumbers": correct,
        "PredictionCount": len(prediction_numbers),
        "NumericalFaithfulness": score
    }


# ============================================================
# Evaluation
# ============================================================

results = []

for p in predictions:

    result = numerical_faithfulness(
        p["input"],
        p["prediction"]
    )

    results.append(result)

df = pd.DataFrame(results)

print("=" * 60)
print("Average Numerical Faithfulness:",
      df["NumericalFaithfulness"].mean())
print("=" * 60)

df.to_csv(
    "NumericalFaithfulness_Qwen-2.5-1.5B.csv",
    index=False,
    encoding="utf-8-sig"
)
```

```
============================================================
Average Numerical Faithfulness: 0.8212745581630282
============================================================
```

Explanation Completeness

In [9]:
```
import re

# ============================================================
# Keywords
# ============================================================

SUPPOR_KEYWORDS = [
    "support",
    "supports",
```

```
    "support threshold",
    "support exceeds",
    "purchased frequently",
    "frequently occurring",
    "frequent itemset"
]

AVG_LENGTH_KEYWORDS = [
    "average support-transaction length",
    "average transaction length",
    "longer transaction",
    "shorter transaction",
    "longer transactions",
    "shorter transactions",
    "average length"
]

FI_SCORE_KEYWORDS = [

    "fi score",

    "score",

    "overall performance",

    "better performance",

    "higher score",

    "lower score",

    "overall score",

    "business impact",

    "overall impact"

]


# ============================================================
# Utility
# ============================================================

def contains_keywords(text, keywords):

    text = text.lower()

    return any(keyword.lower() in text for keyword in keywords)


# ============================================================
# Support
# ============================================================

def evaluate_support(prediction):
```

```
    return int(
        contains_keywords(
            prediction,
            SUPPOR_KEYWORDS
        )
    )


# ============================================================
# Average Support Length
# ============================================================

def evaluate_average_length(prediction):

    return int(
        contains_keywords(
            prediction,
            AVG_LENGTH_KEYWORDS
        )
    )


# ============================================================
# FI Score
# ============================================================

def evaluate_fi_score(prediction):

    return int(
        contains_keywords(
            prediction,
            FI_SCORE_KEYWORDS
        )
    )


# ============================================================
# Overall
# ============================================================

def explanation_completeness(prediction):

    support = evaluate_support(prediction)

    avg = evaluate_average_length(prediction)

    fi = evaluate_fi_score(prediction)

    return {
        "support": support,
        "average_length": avg,
        "fi_score": fi,
        "ExplanationCompleteness":
            (support + avg + fi) / 3
    }
```

```
import pandas as pd

rows = []

for sample in predictions:

    rows.append(
        explanation_completeness(
            sample["prediction"]
        )
    )

df = pd.DataFrame(rows)

# print(df.head())

# print()

print("="*50)
print(df.mean())

df.to_csv(
    "./Training_Data/ExplanationCompleteness_Qwen2.5-1.5B-Instructi
    index=False
)
```

```
==================================================
support                    1.000000
average_length             0.950413
fi_score                   0.644628
ExplanationCompleteness    0.865014
dtype: float64
```

Numerical corectness

In [10]:
```
import re
import pandas as pd

# ============================================================
# CONFIG
# ============================================================

TOLERANCE = 0.1

# ============================================================
# Remove Itemset IDs
# ============================================================

def remove_itemsets(text):
    """
    Remove itemset IDs.

    Example:
        [39, 41, 1799]
```

```
        [6009]
    """

    if text is None:
        return ""

    return re.sub(r"\[[^\]]*\]", "", text)


# ============================================================
# Extract Numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all numerical values.
    """

    text = remove_itemsets(text)

    pattern = r"-?\d+(?:\.\d+)?"

    numbers = re.findall(pattern, text)

    return [float(x) for x in numbers]


# ============================================================
# Direct Match
# ============================================================

def is_direct_match(pred, input_numbers):

    for x in input_numbers:

        if abs(pred - x) <= TOLERANCE:
            return True

    return False


# ============================================================
# Division Match
# ============================================================

def is_division_match(pred, input_numbers):

    n = len(input_numbers)

    for i in range(n):

        for j in range(n):

            if i == j:
                continue
```

```
            a = input_numbers[i]
            b = input_numbers[j]

            # if abs(b) <= TOLERANCE:
            #     continue

            ratio = a / b

            if abs(pred - ratio) <= TOLERANCE:
                return True

    return False


# ============================================================
# Numerical Correctness
# ============================================================

def numerical_correctness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)

    pred_numbers = extract_numbers(prediction_text)

    direct_match = 0
    division_match = 0
    incorrect = 0

    for pred in pred_numbers:

        # 1. Có trực tiếp trong input
        if is_direct_match(pred, input_numbers):

            direct_match += 1

        # 2. Không có nhưng bằng phép chia
        elif is_division_match(pred, input_numbers):

            division_match += 1

        # 3. Sai hoàn toàn
        else:

            incorrect += 1

    correct = direct_match + division_match

    if len(pred_numbers) == 0:
        score = 0
    else:
        score = correct / len(pred_numbers)

    return {

        "input_numbers": len(input_numbers),
```

```
        "predicted_numbers": len(pred_numbers),

        "direct_match": direct_match,

        "division_match": division_match,

        "correct": correct,

        "incorrect": incorrect,

        "NumericalCorrectness": score
    }


# ============================================================
# Evaluate
# ============================================================

rows = []

for sample in predictions:

    rows.append(

        numerical_correctness(
            sample["input"],
            sample["prediction"]
        )

    )

df = pd.DataFrame(rows)

print(df)

print("="*70)
print("Average")
print("="*70)
print(df.mean(numeric_only=True))

print()

print("="*70)
print("Percentage")
print("="*70)

print(f"Average numbers in Input      : {df['input_numbers'].mean()
print(f"Average numbers in Prediction : {df['predicted_numbers'].me
print(f"Direct Match                  : {df['direct_match'].mean():
print(f"Division Match                : {df['division_match'].mean(
print(f"Correct Numbers               : {df['correct'].mean():.2f}"
print(f"Incorrect Numbers             : {df['incorrect'].mean():.2f
print(f"Numerical Correctness         : {df['NumericalCorrectness']

df.to_csv(
    f"./Training_Data/NumericalCorrectness_Qwen2.5-1.5B-Instruction
```

```
    index=False
)

print("\nSaved to NumericalCorrectness.csv")
```

```
     input_numbers  predicted_numbers  direct_match  division_match
correct  \
0               16                  7             7               0
7
1                8                  7             6               1
7
2               16                  5             3               1
4
3               16                  5             5               0
5
4                8                  3             3               0
3
..             ...                ...           ...             ...
...
116             17                  6             6               0
6
117             16                  7             7               0
7
118              8                  9             7               2
9
119              8                  5             5               0
5
120              8                  7             5               1
6

     incorrect  NumericalCorrectness
0            0              1.000000
1            0              1.000000
2            1              0.800000
3            0              1.000000
4            0              1.000000
..         ...                   ...
116          0              1.000000
117          0              1.000000
118          0              1.000000
119          0              1.000000
120          1              0.857143

[121 rows x 7 columns]
================================================================
==
Average
================================================================
==
input_numbers           12.181818
predicted_numbers        6.330579
direct_match             5.371901
division_match           0.595041
correct                  5.966942
incorrect                0.363636
NumericalCorrectness     0.952642
```

```
dtype: float64

================================================================
==
Percentage
================================================================
==
Average numbers in Input      : 12.18
Average numbers in Prediction : 6.33
Direct Match                  : 5.37
Division Match                : 0.60
Correct Numbers               : 5.97
Incorrect Numbers             : 0.36
Numerical Correctness         : 95.26%

Saved to NumericalCorrectness.csv
```

In [83]:
```python
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ---------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
bert_scores = [0.8343,
               0.7989,
               0.8125,
               0.7862,
               0.8353, 0.9221,
               0.8362, 0.9207  ]   # <-- replace

df = pd.DataFrame({"Model": models, "BERT Score": bert_scores})

# ---------------------- Plot ---------------------------------------
plt.figure(figsize=(10, 6))
bars = plt.bar(df["Model"], df["BERT Score"], color="#4e73df")

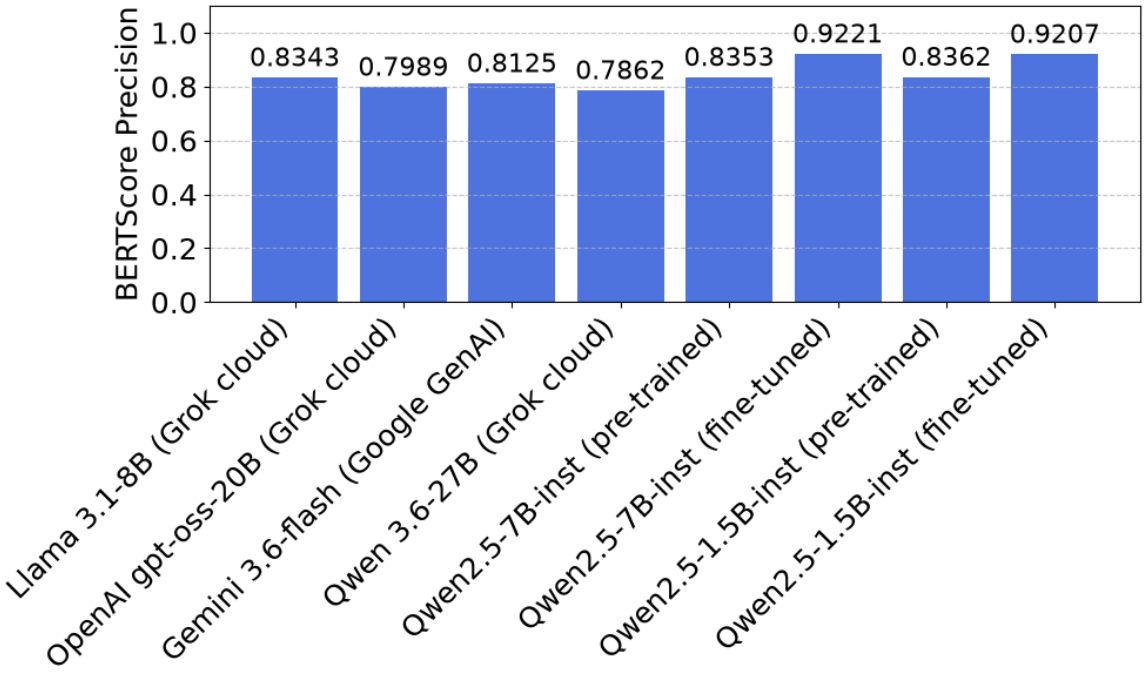
# Font-size tuning
# plt.xlabel("Model", fontsize=18)
plt.ylabel("BERTScore Precision", fontsize=18)
# plt.title("Average BERT Score Across Models", fontsize=20)
plt.xticks(rotation=45, ha="right", fontsize = 18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
```

```
                xytext=(0, 3), textcoords="offset points",
                ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/bert_score_precision.pdf", dpi=300, bb
plt.show()
```




In [84]:

```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ---------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
bert_scores = [0.8610,
               0.8529,
               0.8617,
            0.8485,
            0.8548, 0.9233,
            0.8538, 0.9276
                 ]   # <-- replace

df = pd.DataFrame({"Model": models, "BERT Score": bert_scores})

# ---------------------- Plot ---------------------------------------
```

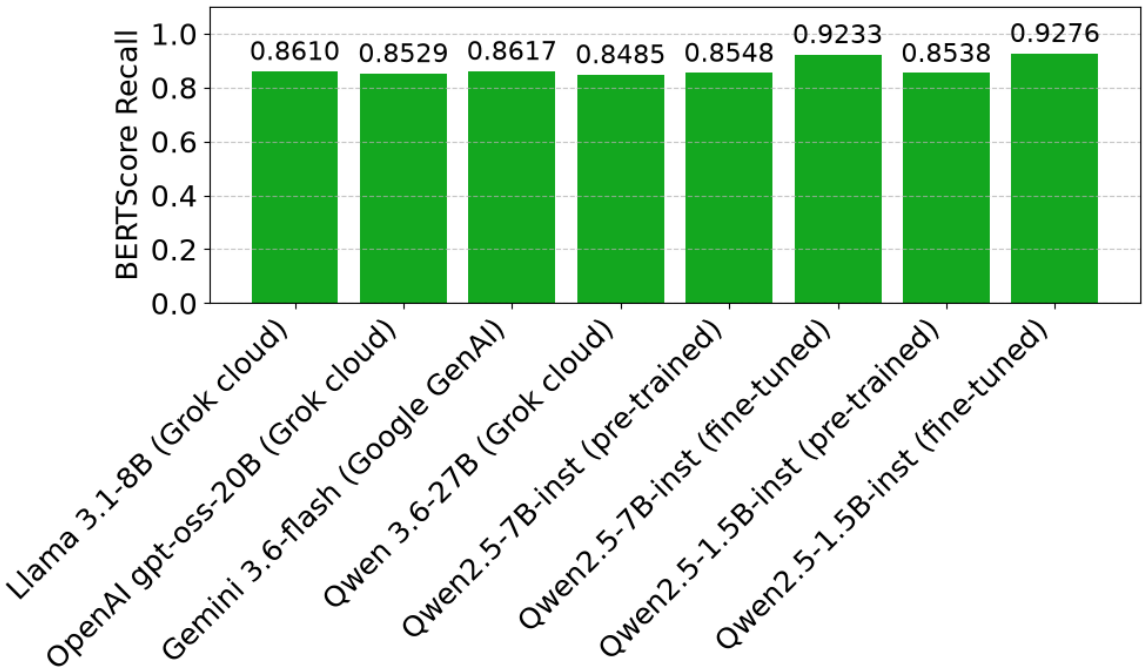
```
plt.figure(figsize=(10, 6))
bars = plt.bar(df["Model"], df["BERT Score"], color="#13a71f")

# Font-size tuning
# plt.xlabel("Model", fontsize= 18)
plt.ylabel("BERTScore Recall", fontsize=18)
# plt.title("Average BERT Score Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize=18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/bert_score_recall.pdf", dpi=300, bbox_
plt.show()
```




In [85]:

```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
```

```
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
bert_scores = [0.8472,
               0.8249,
               0.8362,
               0.8160,
               0.8447, 0.9223,
               0.8447, 0.9238
                 ]   # <-- replace

df = pd.DataFrame({"Model": models, "BERTScore F1": bert_scores})

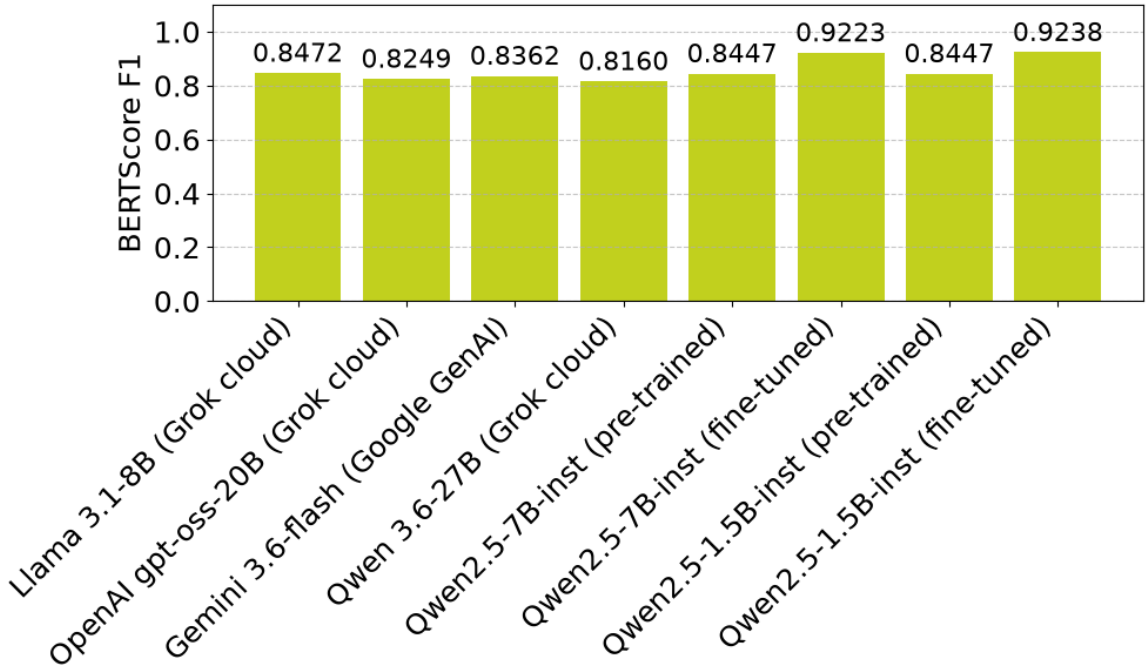
# ---------------------- Plot ------------------------------------
plt.figure(figsize=(10, 6))
bars = plt.bar(df["Model"], df["BERTScore F1"], color="#c1d01e")

# Font-size tuning
# plt.xlabel("Model", fontsize=18)
plt.ylabel("BERTScore F1", fontsize=18)
# plt.title("Average BERT Score Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize=18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/bert_score_f1.pdf", dpi=300, bbox_inch
plt.show()
```




In [86]:
```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.1561,
                  0.1508,
                    0.1807,
                      0.0529,
                        0.2490, 0.6093,
                            0.2426, 0.5875
                ]   # <-- replace

df = pd.DataFrame({"Model": models, "ROUGE-L Precision": rouge_l_sc

# ---------------------- Plot ------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
    "+",
    "x",
    "o",
    "."
```

```
]
bars = plt.bar(df["Model"], df["ROUGE-L Precision"], color="#499ae0

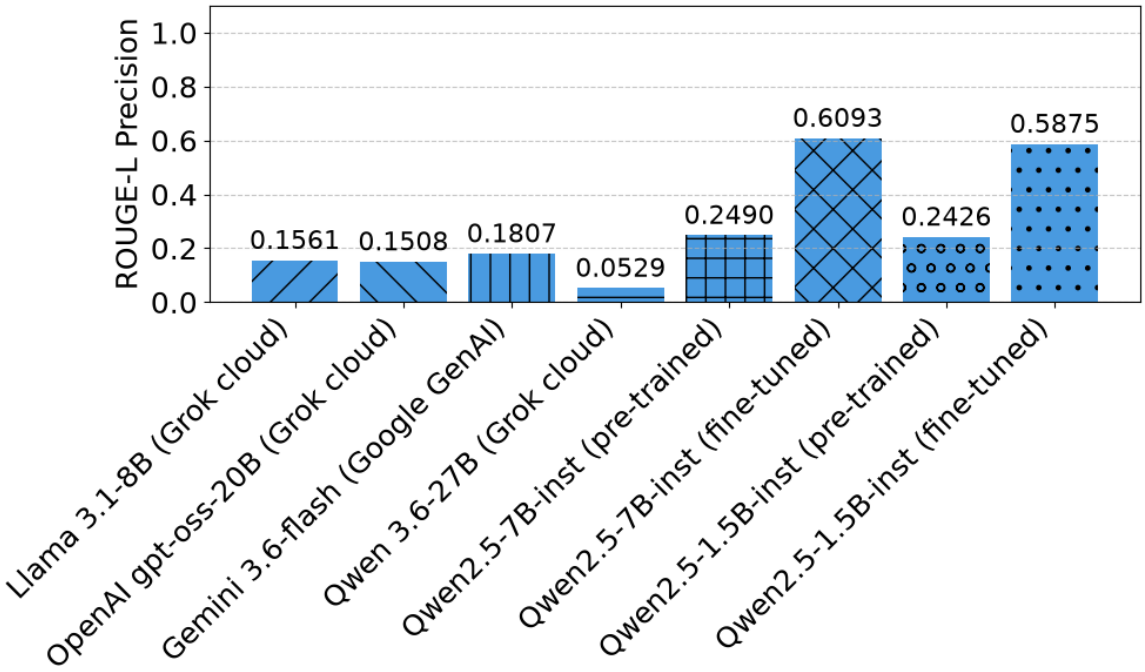
# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("ROUGE-L Precision", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize=18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/rouge_l-Precision.pdf", dpi=300, bbox_
plt.show()
```




In [87]:
```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data -------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
```

```
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.4301,
                  0.3912,
                  0.3897,
                  0.5473,
                  0.3184, 0.6128,
                  0.2782, 0.5873]

df = pd.DataFrame({"Model": models, "ROUGE-L (Recall)": rouge_l_sco

# ---------------------- Plot -------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
    "+",
    "x",
    "o",
    "."
]
bars = plt.bar(df["Model"], df["ROUGE-L (Recall)"], color="#b8464b"

# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

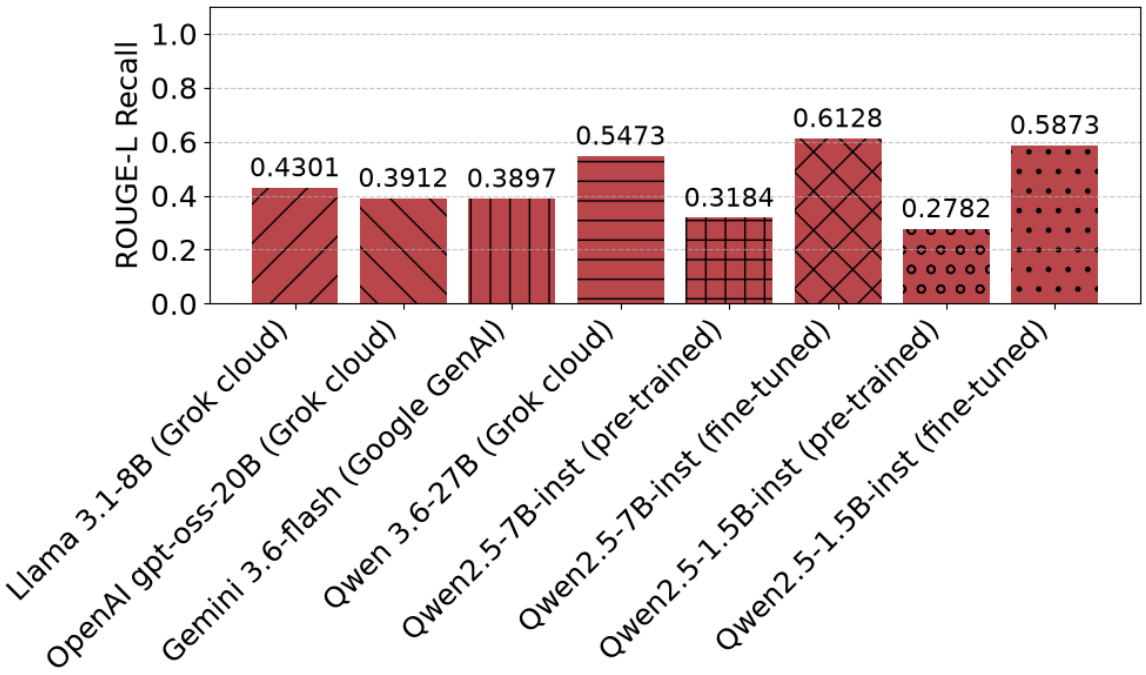
# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("ROUGE-L Recall", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize=18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
```

```
plt.savefig("./Training_Data/rouge_l-Recall.pdf", dpi=300, bbox_inc
plt.show()
```




In [88]:

```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.2186,
                  0.2005,
                  0.2311,
                  0.0946,
                  0.2549, 0.5712,
                  0.2274, 0.5441]

df = pd.DataFrame({"Model": models, "ROUGE-L (F1)": rouge_l_scores}

# ---------------------- Plot ------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
    "+",
    "x",
```

```
    "o",
    "."
]
bars = plt.bar(df["Model"], df["ROUGE-L (F1)"], color="#217718")

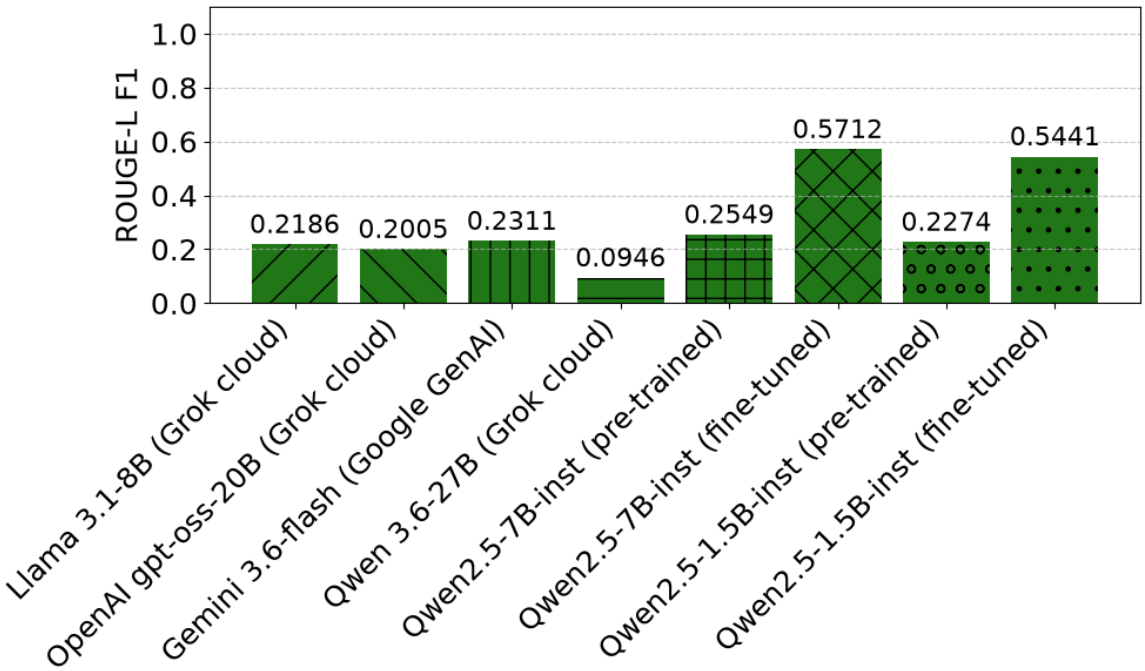
# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("ROUGE-L F1", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize= 18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/rouge_l-F1.pdf", dpi=300, bbox_inches=
plt.show()
```




In [92]:
```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ---------------------------------------
```

```
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.8153,
                  0.4791,
                  0.5308,
                  0.5556,
                  0.7852, 0.7920,
                  0.7841, 0.8212]

df = pd.DataFrame({"Model": models, "Numerical Faithfulness": rouge

# ---------------------- Plot ------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
    "+",
    "x",
    "o",
    "."
]
bars = plt.bar(df["Model"], df["Numerical Faithfulness"], color="#c

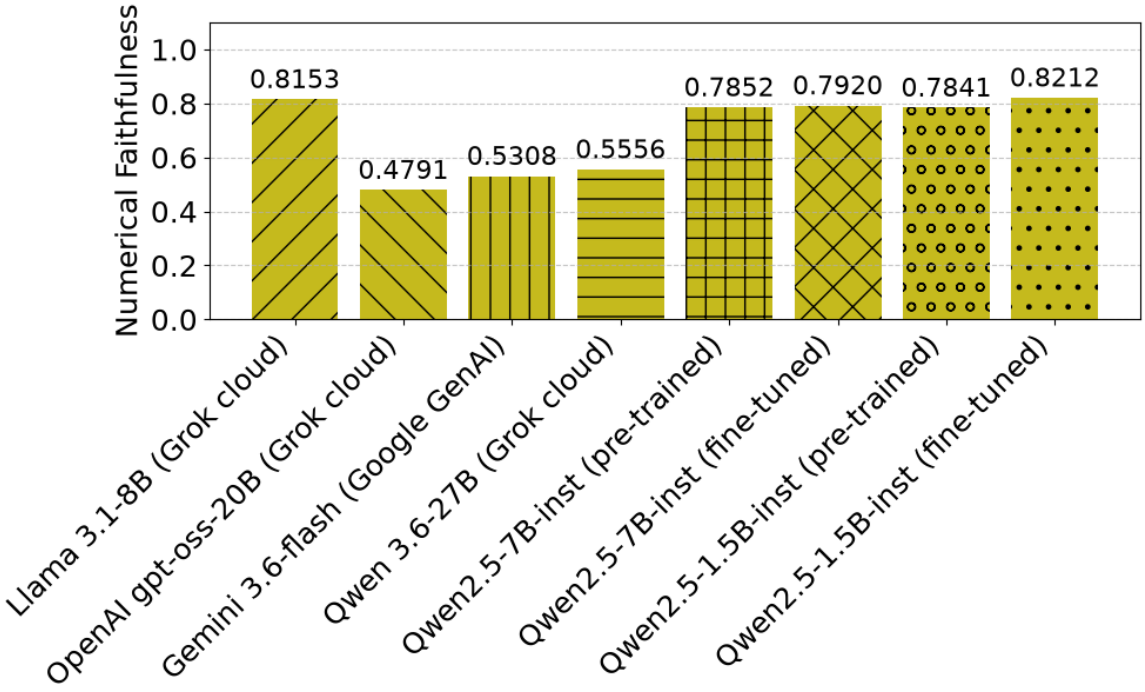
# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("Numerical Faithfulness", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize=18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()
```

```
# Save and show
plt.savefig("./Training_Data/numerical_faithfulness.pdf", dpi=300,
plt.show()
```




In [90]:

```
import matplotlib.pyplot as plt
import pandas as pd

# ---------------------- Data ----------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.8919,
                  0.8381,
                  0.8587,
                  0.9130,
                  0.9055, 0.9204,
                  0.8467, 0.9526]

df = pd.DataFrame({"Model": models, "Numerical correctness": rouge_

# ---------------------- Plot ----------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
```

```
    "+",
    "x",
    "o",
    "."
]
bars = plt.bar(df["Model"], df["Numerical correctness"], color="#3c

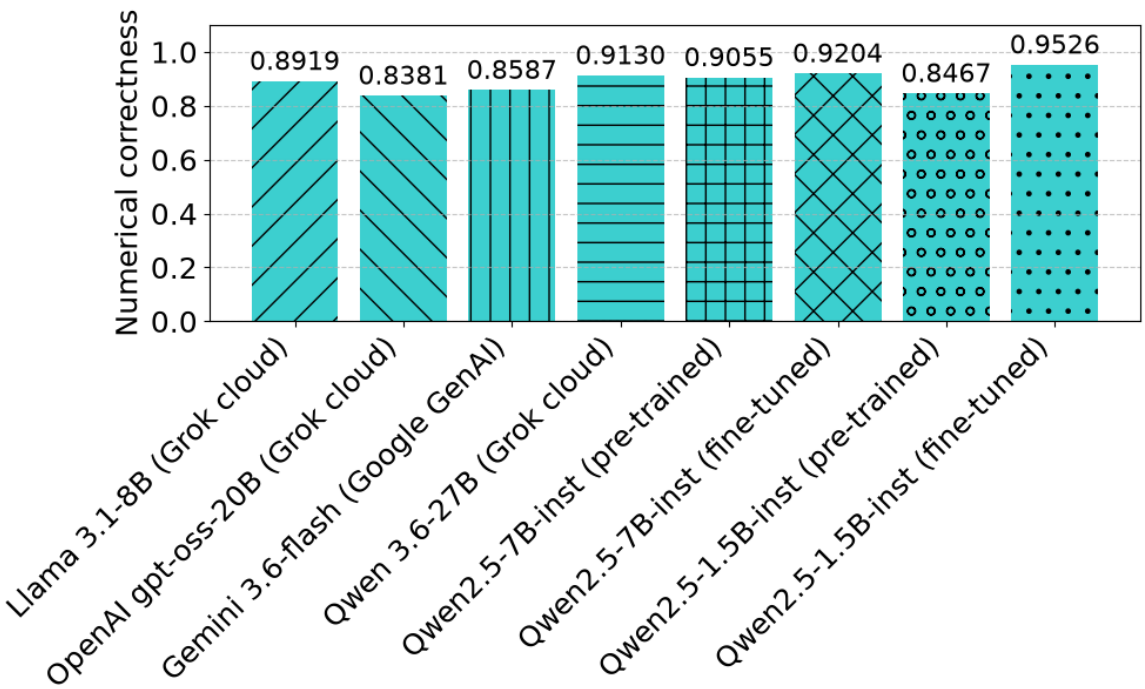
# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("Numerical correctness", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize= 18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)

plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/numerical_correctness.pdf", dpi=300, b
plt.show()
```




In [91]:
```
import matplotlib.pyplot as plt
import pandas as pd
```

```
# ---------------------- Data -------------------------------------
models = [
    "Llama 3.1-8B (Grok cloud)",
    "OpenAI gpt-oss-20B (Grok cloud)",
    "Gemini 3.6-flash (Google GenAI)",
    "Qwen 3.6-27B (Grok cloud)",
    "Qwen2.5-7B-inst (pre-trained)",
    "Qwen2.5-7B-inst (fine-tuned)",
    "Qwen2.5-1.5B-inst (pre-trained)",
    "Qwen2.5-1.5B-inst (fine-tuned)"
]
rouge_l_scores = [0.9449,
                  0.9063,
                  0.8127,
                  0.9367,
                  0.6612,
                  0.7714,
                  0.7934,
                  0.8650]

df = pd.DataFrame({"Model": models, "Explanation completeness": rou

# ---------------------- Plot -------------------------------------
plt.figure(figsize=(10, 6))
# Pattern cho từng cột
patterns = [
    "/",
    "\\",
    "|",
    "-",
    "+",
    "x",
    "o",
    "."
]
bars = plt.bar(df["Model"], df["Explanation completeness"], color="

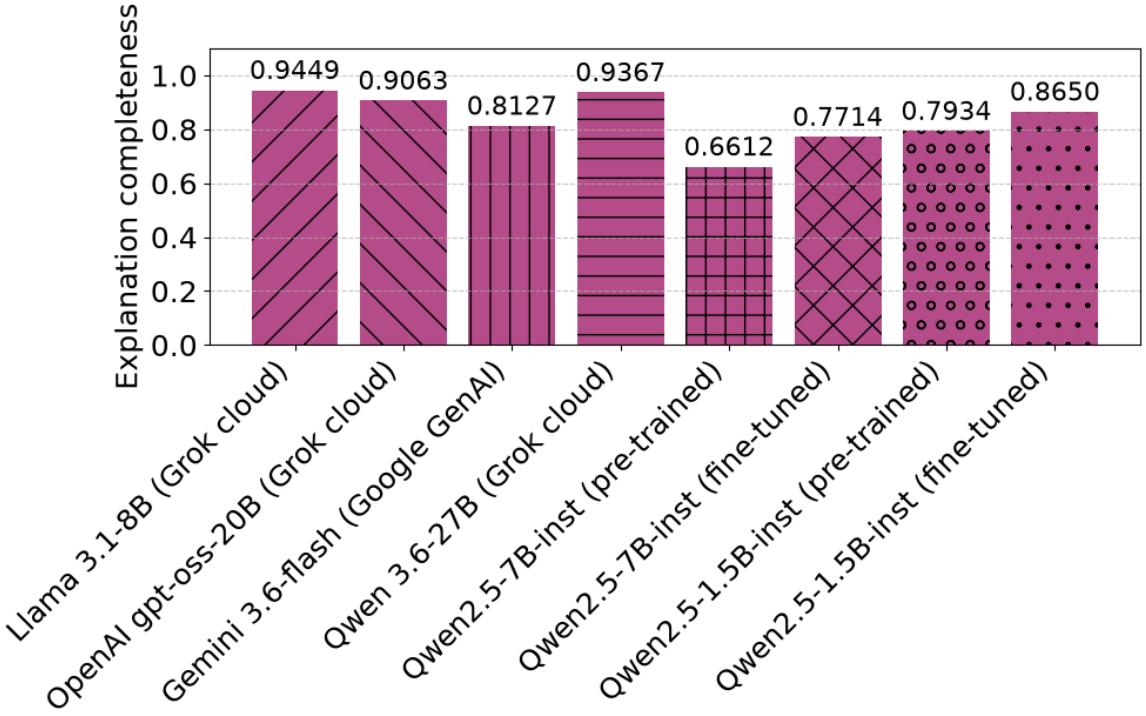
# Áp dụng pattern
for bar, pattern in zip(bars, patterns):
    bar.set_hatch(pattern)

# Font-size tuning
# plt.xlabel("Model", fontsize=16)
plt.ylabel("Explanation completeness", fontsize=18)
# plt.title("Average ROUGE-L Across Models", fontsize=18)
plt.xticks(rotation=45, ha="right", fontsize= 18)
plt.yticks(fontsize=18)

# Annotate bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(f"{height:.4f}",
                 xy=(bar.get_x() + bar.get_width()/2, height),
                 xytext=(0, 3), textcoords="offset points",
                 ha='center', va='bottom', fontsize=16)
```

```
plt.ylim(0, 1.1)
plt.grid(axis="y", linestyle='--', alpha=0.7)
plt.tight_layout()

# Save and show
plt.savefig("./Training_Data/explanation_completeness.pdf", dpi=300
plt.show()
```




In [ ]: